

Student Name: [REDACTED]

ID No.: [REDACTED]

Room no. [REDACTED]

Student Sign.: [REDACTED]

Invigilator Sign.: [REDACTED]

Marks obtained: [REDACTED]

/76

2x	-0.5x	Q.1	Q.2	Q.3	Q.4	Re-check request (if any)
☒	☒	[REDACTED]	[REDACTED]	[REDACTED]	[REDACTED]	☒

[Instructions: There are total 4 questions. Assume that `#include<stdio.h>` is included at the top of each program. Please WRITE YOUR ANSWERS CLEARLY IN THE SPACE PROVIDED. Answers written at any other place will not be evaluated. Use the rough sheet provided for rough work. Note: For MCQs only 1 answer is correct, and there is 25 % negative marking for each wrong answer. If more than 1 answer is written, then that question will not be evaluated.]

[2 marks questions] [For Q. 1, i - v, -0.5 MARKS FOR EACH WRONG ANSWER]

Please tick mark the answer properly. ☒

[10x2=20]

Q.1 Multiple choice questions:

i). Convert (524)₈ to base 8.a. (264)₈ ☐b. (304)₈ ☒c. (312)₈ ☐d. (244)₈ ☐ii). Convert (A3FD)₁₆ to decimal.a. 40981 ☐b. 41981 ☒c. 42981 ☐d. 41891 ☐

iii). What is the output of the following code?

```
int main()
{
    int arr[] = {10, 20, 30, 40};
    int i = 0;
    printf("%d ", arr[i++]);
    printf("%d ", arr[++i]);
    printf("%d", arr[i++]);
    return 0;
}
```

a. 10 30 30 ☒b. 10 20 30 ☐c. 10 30 40 ☐d. 20 30 40 ☐

iv). What is the output of the following code?

```
int main()
{
    int A = 10;
    A = A + 'B';
    printf("%d %d", A, 'A');
    return 0;
}
```

a. 66 65 ☐b. 10 65 ☐c. 77 66 ☐d. 76 65 ☒

v). The value of the 32-bit IEEE 754 FPR 0 10001000 111101010000000000000000 in hexadecimal is _____.

a. 0x447A8000 ☐b. 0x44790018 ☐c. 0x447A8018 ☒d. 0x447B0018 ☐

vi). How many times will "hello" be printed?

```
int main()
{
    int i, j, k;
    for(i = 1; i <= 4; i++)
        for(j = 1; j <= i; j++)
            for(k = 1; k <= j; k++)
                printf("hello\n");
}
```

a. 10 ☐b. 15 ☐c. 20 ☒d. 24 ☐

vii). What is the output of the code?

```
int main()
{
    int i, j;
    for(i = 1; i <= 5; i++) {
        if(i == 3) continue;
        for(j = 1; j <= i; j++) {
            if(j == 2) break;
            printf("*");
        }
    }
}
```

return 0;

}

a. **** ☒b. ***** ☐c. *** ☐d. ***** ☐

viii). What is the output of the code?

```
int main()
{
    int i, j;
    for(i = 1; i <= 4; i++) {
        for(j = 1; j <= 4 - i; j++)
            printf(" ");
        for(j = 1; j <= 2*i - 1; j++)
            printf("*");
        printf("\n");
    }
    return 0;
}
```

- a. Right-aligned triangle ☐ b. Centered pyramid ☒ c. Inverted triangle ☐ d. Inverted pyramid ☐

ix). What is the output of the code?

```
int main()
{
    int a = 5, b = 3, c = 2;
    int result = a > b ? (b > c ? b : c) : a;
    printf("%d", result);
    return 0;
}
```

- a. 5 ☐ b. 2 ☐ c. Compilation error ☐ d. 3 ☒

x). What is the output of the code?

```
#include<string.h>
int main() {
    char a[15];
    char *s = "Hello BITS";
    int len = strlen(s);
    for(int i = 0; i < len; i++)
        a[i] = s[len - 1 - i];
    a[len] = '\0';
    printf("%s", a);
    return 0;
}
```

- a. Hello BITS ☐ b. olleH STIB ☐ c. BITS Hello ☐ d. STIB olleH ☒

[6 marks questions] [For Qs. 2, i-v there is no negative marking. Please note that the outputs are case-sensitive]

Q.2. i). Given the output of the following code is "1 2 4 5 7 9 10 6", fill in the blanks ()

[3x2=6]

```
#include<stdlib.h>
void mysort(int a[], int SIZE) {
    int temp, i, j, sorted = 0;
    for(i = 0; i < SIZE; i++){
        for(j = SIZE-1; j > i; j--){
            if(a[j-1] > a[j])
            {
                temp = a[j];
                a[j] = a[j-1];
                a[j-1] = temp;
            }
        }
    }
}
int main() {
    int arr[8] = {2, 5, 9, 7, 1, 10, 4, 6};
    int SIZE = 8;
    int i;
    mysort(arr, SIZE);
    for (i = 0; i < SIZE; i++)
        printf("%d ", arr[i]);
    return 0;
}
```

a). temp = a[j];
 b). a[j] = a[j-1];
 c). a[j-1] = temp;
 }
 }
 }
 }
 }
 }

ii). Fill the blanks for the following code? Output:

40 50 20 40 40

[3x2=6]

```
int main() {
    int arr[] = {10, 20, 30, 40, 50};
    arr[2] = *(arr + 1);
    *(arr + 1) = ????????;
    int *p;
    p = &arr[5];
    ????????
    ????????
    *arr = *p;
    for (int i = 0; i < 5; i++)
        printf("%d ", *(arr + i));
    return 0;
}
```

//a). arr[4]
 //b). p--;
 //c). *p = *(arr+3);

iii). What is the output of the following code?

```
#define MAX(A, B) ((A > B) ? (A) : (B))
int main() {
    int max_val, x, y;
    x = 5;
    y = 10;
    max_val = MAX(x++, y++);
    printf("The result is: %d\n", max_val);
    printf("x is %d\n", x);
    printf("y is %d", y);
    return 0;
}
```

The result is: 10
 //a). 2 is 6
 //b). 11
 //c). 11

iv). What is the output of the following code?

[3x2=6]

```
int main()
{
    int arr[] = {10, 12, 14};
    int *p, **pp;
    p = arr;
    pp = &p;
    printf("%d, %d\n", *(p + 1), *(*pp + 2));
    printf("%d, %d\n", *p + 1, **pp + 2);
    printf("%d", (*pp)[1]);
    return 0;
}
```

//a). 12, 14 ✓
 //b).
 //c). N.A

2

v). Fill all the blanks (1 mark will be awarded for each row only if entire row is correct, otherwise 0 marks for that row).

[6x1=6]

If File Handling Mode is:	Must the file exist? (Yes/No)	Can it create a file? (Yes/No)	Can it truncate a file? (Yes/No)	Read-ability (Yes/No)	Write-ability (Yes/No)	Append-ability (Yes/No)
"r"	Yes	No	No	Yes	No	No
"r+"	Yes	No	No	Yes	Yes	No
"w"	No	Yes	Yes	No	Yes	No
"w+"	No	Yes	Yes	Yes	Yes	No
"a"	No	Yes	No	No	Yes	Yes
"a+"	No	Yes	No	Yes	Yes	Yes

✓
 ✓
 ✓
 ✓
 ✓
 ✓

6

Q.3. Fill the blanks in the following program.

[7x2=14]

```
Sorted Array 'b':
Output: 1 2 3 4 5
#include <stdlib.h>
void sortAndCopy(int c[], int n, int **d) {
    *d = (int *) malloc(????????);
    if (*d == NULL) {
        printf("Memory allocation failed!\n");
        return;
    }
    for (int i = 0; i < n; i++) {
        (*d)[i] = c[i];
    }
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            // Compare adjacent elements
            if (????????) {
                ??????;
                (*d)[j] = ??????;
                ??????;
            }
        }
    }
}
```

a). sizeof(int) (extra)
 *sizeof(int)

b).
 c).
 d).
 e).
 (*d)[j] > (*d)[j+1]
 int temp = (*d)[j];
 (*d)[j+1] = (*d)[j];
 (*d)[j] = temp

12

```
int main() {
    int a[5] = {1, 5, 4, 2, 3};
    int **b = (int **) malloc(sizeof(int *));
    sortAndCopy(a, 5, b);
    printf("Sorted Array 'b':\n");
    for(int i = 0; i < 5; i++) {
        printf("%d\t", (*b)[i]);
    }
    printf("\n");
    free(?????);
    free(?????);
    return 0;
}
```

f).
 g).
 *b
 **b

Q.4. Fill in the blanks for the following code.

[6x2=12]

```
List: 20 -> 10 -> NULL
Output: List: 10 -> NULL
#include <stdlib.h>
typedef struct dllnode * DLLNODE;
struct dllnode{
    int ele;
    DLLNODE next;
    DLLNODE prev;};
```



```

typedef struct doubly_linked_list * DLIST;
struct doubly_linked_list{
    int count;
    DLLNODE head;
};

DLIST createNewList(){
    DLIST myList;
    myList = (DLIST) malloc(sizeof(struct doubly_linked_list));
    myList->count=0;
    myList->head=NULL;
    return myList;
}

DLLNODE createNewNode(int value){
    DLLNODE myNode;
    myNode = (DLLNODE) malloc(sizeof(struct dllnode));
    ????????
    ????????
    ????????
    return myNode;
}

void insertNodeIntoList(DLLNODE n1, DLIST l1){
    // case when list is empty
    if(l1->count == 0) {
        l1->head = n1;
        n1->next = NULL;
        n1->prev = NULL;
        l1->count++;
    }
    else { // case when list is non empty. Insert at beginning
        n1->next = l1->head;
        n1->prev = NULL;
        n1->next->prev = n1;
        ????????
        l1->count++;
    }
}

void removeFirstNode(DLIST l1)
{
    if(l1->count==0)
    {printf("List is empty. Nothing to remove\n");}
    else
    {
        DLLNODE temp=l1->head;
        ?????????
        l1->head->prev = NULL;
        free(temp);
        l1->count--;
    }
    return;
}

void printList(DLIST l1) {
    if (l1->count == 0) {
        printf("List is empty\n");
    } else {
        DLLNODE temp = l1->head;
        printf("List: ");
        while (temp != NULL) {
            printf("%d -> ", temp->ele);
            ????????
        }
        printf("NULL\n");
    }
}

int main()
{
    DLIST newList = createNewList();
    // Create new nodes
    DLLNODE n1 = createNewNode(10);
    DLLNODE n2 = createNewNode(20);

    insertNodeIntoList(n1, newList);
    insertNodeIntoList(n2, newList);
    printList(newList);
    removeFirstNode(newList); // Remove first node and print list
    printList(newList);
    return 0;
}

```

a). myNode->ele = value; ✓
 b). myNode->next = NULL; ✓
 c). myNode->prev = NULL; ✓

(12)

d). l1->head = n1 ✓

e). l1->head = temp->next ✓

f). temp = temp->next ✓

Handwritten calculations and diagrams:

- Diagram 1: A linked list with nodes 10 and 20. Node 10 points to Node 20. Node 20 points to NULL. The list is represented as 10 → 20 → NULL.
- Diagram 2: A linked list with nodes 10 and 20. Node 10 points to Node 20. Node 20 points to NULL. The list is represented as 10 → 20 → NULL.
- Diagram 3: A linked list with nodes 10 and 20. Node 10 points to Node 20. Node 20 points to NULL. The list is represented as 10 → 20 → NULL.
- Diagram 4: A linked list with nodes 10 and 20. Node 10 points to Node 20. Node 20 points to NULL. The list is represented as 10 → 20 → NULL.
- Diagram 5: A linked list with nodes 10 and 20. Node 10 points to Node 20. Node 20 points to NULL. The list is represented as 10 → 20 → NULL.

END OF QUESTION PAPER